

NOW LETS GET STARTED-----

WITH THE PRESENTATION.

makeameme.org

Object Oriented Programming (OOP)

Object Oriented Programming (OOP) is a programming approach that focuses on objects rather than functions.

An object represents a real-world entity such as a student, car, or employee.

OOP helps in organizing complex programs into small, manageable parts.

It improves code reusability, security, and maintainability.

Languages that support OOP include C++, Java, Python, and C#.

Basic Concepts of OOP

a) Class

A class is a blueprint or template used to create objects.

It defines data members and member functions.

It contains:

Data members (variables)

Data members are variables defined inside a class that store data related to an object.

Member functions (methods)

Member functions are functions declared inside a class that operate on the data members of that class

General Syntax:

cpp

```
class ClassName
{
    access_modifier:
        data_members;

        member_functions;
};
```

b) Object

An object is an instance of a class.

It represents a real-world entity and uses class features.

General Syntax:

```
cpp
```

```
ClassName objectName;
```

Objects access class members using the dot (.) operator.

Multiple objects can be created from the same class.

Each object has its own copy of data members.

General syntax:(to access member)

```
cpp
```

```
objectName.memberName;
```

Example:

```
try.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  class Student
5  {
6      int rollNo;
7      float marks;
8
9      Tabnine | Edit | Test | Explain | Document
10     void input()
11     {
12         cin >> rollNo >> marks;
13     }
14
15     Tabnine | Edit | Test | Explain | Document
16     void display()
17     {
18         cout << "Roll No: " << rollNo << endl;
19         cout << "Marks: " << marks << endl;
20     }
21 };
22
23 Tabnine | Edit | Test | Fix | Explain | Document
24 int main()
25 {
26     Student s1;
27     s1.input(); // Not accessible
28     s1.display(); // Not accessible
29
30     return 0;
31 }
```

Sweet error:

```
Aamir@Aamir-PC MINGW64 /d/Emad data/emad data/Codealpha/c++ language/oop/oopp (cla
⊙ $ g++ ./try.cpp && ./a.exe
./try.cpp: In function 'int main()':
./try.cpp:24:14: error: 'void Student::input()' is private within this context
    s1.input();    // â?0 Not accessible
        ^
./try.cpp:9:10: note: declared private here
    void input()
        ^~~~~~
./try.cpp:25:16: error: 'void Student::display()' is private within this context
    s1.display(); // â?0 Not accessible
        ^
./try.cpp:14:10: note: declared private here
    void display()
        ^~~~~~
```


Explanation of this error:

This error occurs because class members are private by default when no access modifier is used, so they cannot be accessed outside the class.

Access Modifiers

- **Access modifiers** control the visibility and accessibility of class members.
- They define **where data members and member functions can be accessed** from.
- Access modifiers are used to **protect data** and implement encapsulation.
- In C++, there are **three access modifiers**.

Access modifiers are keywords that control the accessibility of class members in a program.

Types of Access Modifiers in C++

1. Public

- Members are accessible **from anywhere** in the program.
- They can be accessed using an object of the class.
- Commonly used for functions that interact with users.

cpp

```
public:  
    int rollNo;
```

2. Private

- Members are accessible **only inside the class**.
- This is the **default access modifier** in a class.
- Used to protect sensitive data.

cpp

```
private:  
    int marks;
```

3. Protected

- Members are accessible inside the class and in **derived (child) classes**.
- Not accessible directly through objects.

```
cpp
```

```
protected:
```

```
    int age;
```

Example:

```
try.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class Student {
5  public:
6      int rollNo;
7      string name;
8
9      void display() {
10         cout << "Roll No: " << rollNo << ", Name: " << name << endl;
11     }
12 };
13
14 int main() {
15     Student s1; // object creation
16     s1.rollNo = 101;
17     s1.name = "Ali";
18     s1.display();
19
20     Student s2;
21     s2.rollNo = 102;
22     s2.name = "Sara";
23     s2.display();
24
25     return 0;
26 }
27
```

Result:

```
Aamir@Aamir-PC MINGW64 /d/Emad
$ g++ ./try.cpp && ./a.exe
Roll No: 101, Name: Ali
Roll No: 102, Name: Sara

Roll No: 101, Name: Ali
Roll No: 102, Name: Sara

Roll No: 102, Name: Sara
```

Encapsulation:

Encapsulation is the concept of **binding data (variables) and functions (methods) together** into a single unit called a **class** and **restricting direct access** to some of the class members to protect data.

How it works:

- **Private members** → store the actual data, hidden from outside.
- **Public members** → provide **controlled access** to the private data through **functions** (like setters and getters).

Example:

```
try.cpp > ...
1  #include <iostream>
2  using namespace std;
3  class BankAccount {
4  private:
5      float balance; // private data
6  public:
7      void deposit(float amt) {
8          balance += amt;
9      }
10     void withdraw(float amt) {
11         if (amt <= balance)
12             balance -= amt;
13         else
14             cout << "Not enough balance!" << endl;
15     }
16     void display() {
17         cout << "Balance: " << balance << endl;
18     }
19     void setBalance(float b) { // set initial balance
20         balance = b;
21     }
22 };
23 int main() {
24     BankAccount acc;
25     acc.setBalance(5000); // initial balance
26     acc.deposit(2000);    // deposit money
27     acc.withdraw(1500);   // withdraw money
28     acc.display();        // show final balance
29     return 0;
30 }
```

Constructors in C++

A **constructor** is a **special member function** of a class that is automatically called **when an object is created**.

It is used to **initialize data members** of the object.

Key Points

1. **Name:** Same as the class name.
2. **No return type** (not even void).
3. **Automatically executed** when an object is created.
4. Can have **parameters** (Parameterized constructor) or no parameters (Default constructor).
5. Helps in **initializing objects easily**.

General Syntax

cpp

```
class ClassName {  
public:  
    ClassName() { // constructor  
        // initialization code  
    }  
};
```

Types of Constructors in C++

Constructors in C++ are **special member functions** used to initialize objects.

Default Constructor

- A constructor **without parameters**.
- Automatically initializes data members with default values.

```
cpp

class Student {
public:
    int rollNo;
    Student() { // default constructor
        rollNo = 101;
    }
};
```

2. Parameterized Constructor

- A constructor **with parameters**.
- Used to initialize objects with **specific values** at the time of creation.

cpp

```
class Student {
public:
    int rollNo;
    float marks;

    // Parameterized constructor
    Student(int r, float m) {
        rollNo = r;
        marks = m;
    }

    void display() {
        cout << "Roll No: " << rollNo << ", Marks: " << marks << endl;
    }
};

int main() {
    Student s1(102, 88.5); // constructor with parameters
    s1.display();

    return 0;
}
```

Copy Constructor

- A constructor that **creates a new object as a copy of an existing object**.
- Syntax: `ClassName(const ClassName &obj)`

try.cpp > main()

```
1  #include <iostream>
2  using namespace std;
3  class Student {
4  public:
5      int rollNo;
6      float marks;
7      // Parameterized constructor
8      Student(int r, float m) {
9          rollNo = r;
10         marks = m;
11     }
12     // Copy constructor
13     Student(const Student &s) {
14         rollNo = s.rollNo;
15         marks = s.marks;
16     }
17     void display() {
18         cout << "Roll No: " << rollNo << ", Marks: " << marks << endl;
19     }
20 };
21 int main() {
22     Student s1(101, 95.5); // original object
23     Student s2 = s1;       // copy constructor called
24     cout << "Original Student: ";
25     s1.display();
26     cout << "Copied Student: ";
27     s2.display();
28     return 0;
29 }
```


Destructor in C++

A **destructor** is a **special member function** of a class that is automatically called **when an object goes out of scope or is destroyed**.

It is used to **release resources** like memory, file handles, or network connections.

Key Points

1. **Same name as the class** but preceded by a **tilde (~)**.
2. **No return type** (not even void).
3. **No parameters**.
4. Automatically called **when an object is destroyed**:
 - At the end of `main()`
 - When a local object goes out of scope
 - When a dynamically allocated object is deleted using `delete`
5. Usually used to **clean up resources** (memory, files, database connections).
6. Each object has **its own destructor**.

A destructor is a special function with a **tilde (~) before the class name**, automatically called when an object is destroyed to **release resources**.

Genral syntax:

cpp

```
class ClassName {  
public:  
    ~ClassName() {    // destructor  
        // code to clean up resources  
    }  
};
```

Example:

cpp

```
#include <iostream>
using namespace std;

class Student {
public:
    int rollNo;

    // Constructor
    Student(int r) {
        rollNo = r;
        cout << "Constructor called for Roll No: " << rollNo << endl;
    }

    // Destructor
    ~Student() {
        cout << "Destructor called for Roll No: " << rollNo << endl;
    }
};

int main() {
    Student s1(101); // constructor called
    Student s2(102); // constructor called

    cout << "End of main function" << endl;
    return 0;
}
```



Result

```
Constructor called for Roll No: 101  
Constructor called for Roll No: 102  
End of main function  
Destructor called for Roll No: 102  
Destructor called for Roll No: 101
```

Inheritance in C++

Inheritance is an OOP concept in which one class **acquires the properties and behavior** (data members and member functions) of another class.

- The class that gives properties is called the **Base (Parent) class**
- The class that receives properties is called the **Derived (Child) class**

Inheritance is the mechanism by which one class acquires the properties and behaviors of another class.

Why Inheritance is Used

- Code **reusability**
- Saves time and reduces repetition
- Makes programs easier to maintain
- Represents **real-world relationships** (IS-A relationship)

General syntax

cpp

```
class DerivedClass : access_modifier BaseClass {  
    // members of derived class  
};
```


Example:

C++ oop.cpp > ...

```
134  #include <iostream>
135  using namespace std;
136
137  // Base class
138  class Person {
139  public:
140      string name;
141
142      void showName() {
143          cout << "Name: " << name << endl;
144      }
145  };
146
147  // Derived class
148  class Student : public Person {
149  public:
150      int rollNo;
151
152      void showRollNo() {
153          cout << "Roll No: " << rollNo << endl;
154      }
155  };
156
157  int main() {
158      Student s1;
159
160      s1.name = "Ali";      // inherited member
161      s1.rollNo = 101;
162
163      s1.showName();        // inherited function
164      s1.showRollNo();
165
166      return 0;
167  }
```

Result:

```
Aamir@Aamir-PC MINGW64 /d/Emad data/emad data
● $ g++ ./oop.cpp && ./a.exe
Name: Ali
Roll No: 101
```

Mode of inheritance:

	Derived Class	Derived Class	Derived Class
Base Class	Private Mode	Protected Mode	Public Mode
Private	Not Inherited	Not Inherited	Not Inherited
Protected	Private	Protected	Protected
Public	Private	Protected	Public

Types of Inheritance

1. **Single Inheritance**
 - One base class → one derived class
2. **Multilevel Inheritance**
 - Class derived from another derived class
3. **Multiple Inheritance**
 - One class inherits from more than one base class

END OF PRESENTATION

**ANY QUESTIONS?
CLARIFICATIONS?**

makeameme.org

